

CSCI 1730 Breakout Lab 04

Bitwise Not, Left Shift, and Right Shift

Learning Outcomes

- Understand how unsigned integers are represented and stored in memory by converting unsigned integers to and from binary to base-10.
- Understand binary representation of integers in memory and use bitwise operators to solve problems.
- Design and implement a program that uses bitwise operators on unsigned integers.
- Trace, design, and implement software solutions to non-trivial problems using the C programming language.
- Design and implement programs that use command line arguments.

Problem / Exercise

The purpose of this lab is to get experience with the bitwise not operator, the left shift operator, and the right shift operator. You will implement a C program that will process command line arguments, as shown in the examples, in two ways:

1. a flag representing bitwise not and a bit string;
2. a bit string, a flag representing a left shift operator or a right shift operator, and an amount to left shift or right shift.

Your C program should output the correct result of the appropriate operator in binary and in base-10 as shown in the examples. The three possible flags for this assignment are `-not` to denote the `~` operator, `-leftshift` to denote the `<<` operator, and `-rightshift` to denote the `>>` operator. Let n be the length of the bit string inputted on the command line. You should assume all of the following for this assignment.

- $1 \leq n \leq 64$.
- All of the bit strings in this assignment represent unsigned integers.
- All amounts to left shift or to right shift will be inputted as an argument that represents a base-10 integer greater than or equal to zero. Also, all amounts to left shift or to right shift will be inputted as an argument that represents a base-10 integer less than or equal to the value of the bit string argument represented in base-10.

Before you write any code for this assignment, you should work through all examples with paper and pencil.

Examples

Your C source code should be in a file called `lab04.c`, and it should be compiled into an executable called `lab04.out`. Your C program must look exactly like the examples below when run on the command line on `odin`, it must compile correctly, and it must run correctly with any valid sequence of inputs provided as command line arguments. You may assume that the command line arguments will be entered in a correct manner as demonstrated by the examples. Each example is a separate execution of a correct program for this assignment.

```
./lab04.out -not 011
```

```
~011 evaluates to 100 using bit strings of length 3
```

```
~3 evaluates to 4 using unsigned 3-bit integers
```

```
./lab04.out -not 1010110001
```

```
~1010110001 evaluates to 0101001110 using bit strings of length 10
```

```
~689 evaluates to 334 using unsigned 10-bit integers
```

```

./lab04.out -not 0
~0 evaluates to 1 using bit strings of length 1
~0 evaluates to 1 using unsigned 1-bit integers

./lab04.out 00110110 -leftshift 4
00110110 << 00000100 evaluates to 01100000 using bit strings of length 8
54 << 4 evaluates to 96 using unsigned 8-bit integers

./lab04.out 1001111 -leftshift 1
1001111 << 0000001 evaluates to 0011110 using bit strings of length 7
79 << 1 evaluates to 30 using unsigned 7-bit integers

./lab04.out 0101 -leftshift 0
0101 << 0000 evaluates to 0101 using bit strings of length 4
5 << 0 evaluates to 5 using unsigned 4-bit integers

./lab04.out 110100000000 -rightshift 6
110100000000 >> 000000000110 evaluates to 000000110100 using bit strings of length 12
3328 >> 6 evaluates to 52 using unsigned 12-bit integers

./lab04.out 10010 -rightshift 3
10010 >> 00011 evaluates to 00010 using bit strings of length 5
18 >> 3 evaluates to 2 using unsigned 5-bit integers

./lab04.out 01 -rightshift 0
01 >> 00 evaluates to 01 using bit strings of length 2
1 >> 0 evaluates to 1 using unsigned 2-bit integers

```

1 C Program

1.1 General Requirements

1. Place your C source code for this assignment in a file named `lab04.c`. See the Examples section to see what your C program should output. Compile and link your C program using the following command. If there are any warnings or errors from the compiler, then you should fix them before you submit this assignment to us for grading.

```
gcc -Wall lab04.c -o lab04.out
```

2. Place the files for this assignment in a directory called `lab04`.
3. Your program's source file(s) must compile on `odin` using the required compiler for this course (`gcc version 11.2.0`).
4. Do *NOT* include `math.h` in your source code. Do *NOT* call any functions in `math.h`.
5. Do *NOT* use a `break` statement, a `continue` statement, or a `goto` statement anywhere in your source code.
6. All written and verbal instructions stated by the teaching staff (lecture instructor, lab instructor(s), etc.) must be followed for this assignment. Failure to follow instructions may result in losing points.

1.2 Coding Style Requirements

1. All functions must be commented. Comments must include a brief description of what the function does, its input(s), its output, and any assumptions associated with calling it. If your function has a prototype and an implementation separated into different parts of your source code, then you only need to put the comments for that function above its prototype (there is no need to comment both the prototype and implementation; commenting the prototype is sufficient).
2. All structs, unions, and enums must be commented.
3. All global variables and static variables must be commented.

4. All identifiers must be named well to denote their functionality. Badly named identifiers are not allowed. For example, identifiers like a, aaa, b, bbb, bbbb are bad names for identifiers.
5. Every line of source code must be indented properly and consistently.

1.3 README.txt File Requirements

Make sure to include a `README.txt` file (use a `.txt` file extension) that includes the following information presented in a reasonably formatted way:

- Your First and Last Name (as they appear on eLC) and your 810/811#
- Instructions on how to compile and run your program.

2 Submission

Submit your files before the due date and due time stated on eLC. Submit your files on eLC under the Assignment named Lab 04. Submit only the following files.

1. All source files required for this lab: `lab04.c`
2. A `README.txt` file filled out correctly

Do not submit any compiled code. Also, do not submit any zipped files or directories. We only need the files mentioned above with the file extensions aforementioned.

3 Breakout Lab Attendance: 1 point

Breakout lab attendance is required for this assignment, and it is worth one point. Students must physically attend the entire period of the breakout lab section that they registered for during the week of this assignment to earn this point for attendance. If you complete the assignment before the end of your breakout lab period during the week of this assignment, then you are still required to attend the entire breakout lab period to earn your attendance point, and you may use this time for independent study for the next exam in this course.

4 Grading: 10 points

If your program does not compile on `odin` using the required compiler for this course (`gcc version 11.2.0`), then you'll receive a grade of zero on this part of the assignment. Otherwise, your program will be graded using the criteria below.

Program runs correctly with various test cases on <code>odin</code>	10 points
<code>README.txt</code> file missing or filled out incorrectly	-1 point
Not submitting to the correct Assignment on eLC	-1 point
Not adhering to one or more coding style requirements	-1 point
Late penalty for submitting 0 hours to 24 hours late	-2 points
One or more compiler warnings	-2 points
Not submitting this assignment before its late period expires	-10 points
Penalty for not following instructions (invalid I/O, etc.)	Penalty decided by grader

You must test, test, and retest your code to make sure it compiles and runs correctly on `odin` with any given set of valid inputs. This means that you need to create many examples on your own (that are different than the aforementioned examples) to ensure you have a correctly working program. Your program's I/O must match the examples given in the Examples section. We will only test your program with valid sets of inputs.